Solution: Basic Calculator

Problem Statement

Given a string `s`, representing the `arithmetic expressions`, evaluate the expression and return the resultant value.

Constraints:

- s contains digits, '+', '-', '(', ')', and ' '.
- '+' is used as addition operation but not as a unary operator. (i.e., "+2" and "+(1 + 3)" is invalid).
- '-' is used as a unary and subtraction operator both. (i.e., "-2" and "-(1 + 3)" is valid).

Note: You are not allowed to use the built-in function like `eval()` to evaluate the mathematical expression.

Examples

- **Example 1**:
  - **Input**: `"3 - (2 - 1) + 4"`
  - **Expected Output**: `6`
  - **Justification**: The expression inside the parentheses `2 - 1` evaluates to `1`. So the expression becomes `3 - 1 + 4`, which sums up to `6`.
- **Example 2**:
  - **Input**: `"10 + (8 - (4 + 2)) - 1"`
  - **Expected Output**: `11`
  - **Justification**: Inside the inner parentheses, `4 + 2` equals `6`. The expression becomes `10 + (8 - 6) - 1`. Then `8 - 6` equals `2`, so it simplifies to `10 + 2 - 1`, which is `11`.
- **Example 3**:
  - **Input**: `"5 + (3 - (2 + (8 - 5)) + 7) - (6 - 4)"`
  - **Expected Output**: `8`
  - **Justification**: In the innermost parentheses, `8 - 5` equals `3`. The expression becomes `5 + (3 - (2 + 3) + 7) - (6 - 4)`. The next inner expression `2 + 3` equals `5`, so it becomes `5 + (3 - 5 + 7) - 2`. Simplifying further to `5 + 5 - 2`, we get `10 - 2`, which is `8`.

Solution

To solve this problem, we utilize a stack-based approach to handle the precedence imposed by parentheses and to perform the calculations. This method is effective as stacks allow us to manage the order of operations and operands in a controlled manner.

Initially, we iterate through the expression, handling digits, operators, and parentheses separately. For digits, we construct numbers; for operators, we manage the current operation; and for parentheses, we use the stack to preserve the state of the calculation. When we encounter an open parenthesis, we push the current result and the sign onto the stack and reset them for the new inner expression. Upon encountering a closing parenthesis, we pop from the stack to restore the state and combine it with the current result. This method ensures that we maintain the correct order of operations and handle nested expressions effectively.

Step-by-Step Algorithm

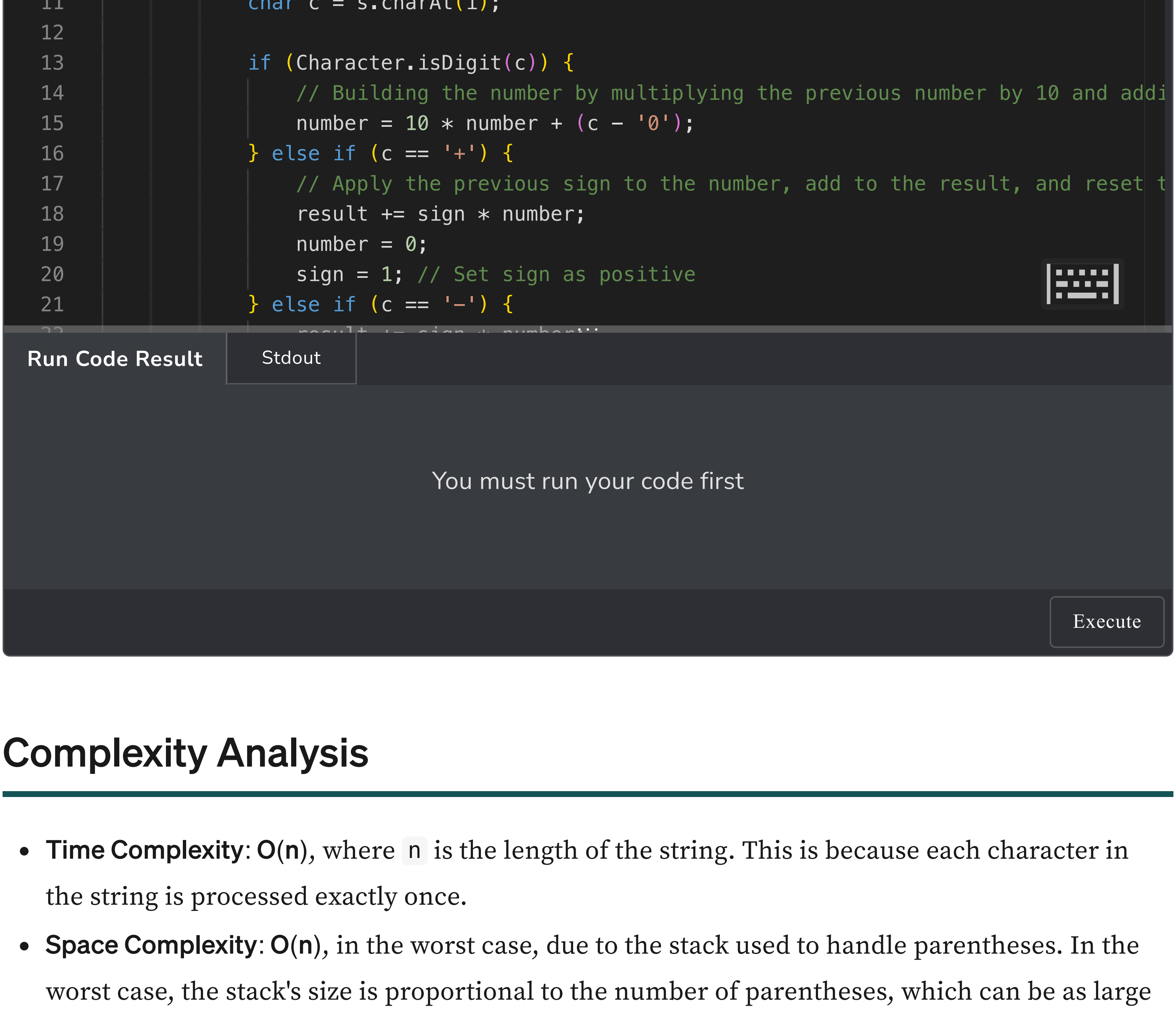
1. **Initialize Variables**:
   - Create a `result` variable to store the cumulative result of the expression.
   - Define a `number` variable to build numbers as they are read from the string.
   - Set a `sign` variable to +1 or -1, indicating the current sign (positive or negative) for the number being processed.
   - Initialize a `stack` to handle parentheses in the expression.
2. **Iterate Over the String**:
   - Loop through each character in the input string.
3. **Process Each Character**:
   - If the character is a digit:
     - Update `number` by multiplying the current `number` by 10 and adding the new digit. This step builds multi-digit numbers from consecutive digits in the string.
   - If the character is a '+' or '-':
     - Apply the previous `sign` to the current `number` and add it to `result`.
     - Reset `number` to 0.
     - Update `sign` based on whether the character is '+' (set `sign` to 1) or '-' (set `sign` to -1).
   - If the character is an opening parenthesis '(':
     - Push the current `result` and `sign` onto the stack. This step saves the current state before processing the expression inside the parentheses.
     - Reset `result` to 0 and `sign` to 1 to start calculating the expression inside the parentheses.
   - If the character is a closing parenthesis ')':
     - First, add the current `number` to `result` after applying the current `sign`.
     - Then, the expression inside the parentheses is complete. Pop the last `sign` from the stack and multiply it with the current `result`.
     - Pop the last saved `result` from the stack and add it to the current `result`.
     - Reset `number` to 0.
4. **Final Calculation**:
   - After the loop, there might be a number that hasn't been added to the `result`. Add this number to `result` after applying the last `sign`.
5. **Return Result**:
   - Return the final `result`, which is the evaluation of the entire arithmetic expression.

5. Algorithm Walkthrough

Let's use the input `"5 + (3 - (2 + (8 - 5)) + 7) - (6 - 4)"`:

1. **Initialize Variables**:
   - `result = 0`, `number = 0`, `sign = 1`, `stack = []`.
2. **Process Each Character**:
   - '**5**': `number = 5`.
   - '**+**': `result = 0 + (5 * 1) = 5`, `number = 0`, `sign = 1`, `stack = []`.
   - '**(**': push `result (5)` and `sign (1)` to `stack`, `stack = [5, 1]`, reset `result = 0` and `sign = 1`.
   - '**3**': `number = 3`.
   - '**-**': `result = 0 + (3 * 1) = 3`, `number = 0`, `sign = -1`, `stack = [5, 1]`.
   - '**(**': push `result (3)` and `sign (-1)` to `stack`, `stack = [5, 1, 3, -1]`, reset `result = 0` and `sign = 1`.
   - '**2**': `number = 2`.
   - '**+**': `result = 0 + (2 * 1) = 2`, `number = 0`, `sign = 1`, `stack = [5, 1, 3, -1]`.
   - '**(**': push `result (2)` and `sign (1)` to `stack`, `stack = [5, 1, 3, -1, 2, 1]`, reset `result = 0` and `sign = 1`.
   - '**8**': `number = 8`.
   - '**-**': `result = 0 + (8 * 1) = 8`, `number = 0`, `sign = -1`, `stack = [5, 1, 3, -1, 2, 1]`.
   - '**5**': `number = 5`.
   - '**)**': `result = 8 + (5 * -1) = 3`, `result *= stack.pop() (1) = 3`, `result += stack.pop() (2) = 5`, `number = 0`, `stack = [5, 1, 3, -1]`.
   - '**)**': `result *= stack.pop() (-1) = -5`, `result += stack.pop() (3) = -2`, number = 0 `,` stack = [5, 1]``.
   - '**+**': `sign = 1`.
   - '**7**': `number = 7`.
   - '**)**': `result = -2 + (7 * 1) = 5`, `result *= stack.pop() (1) = 5`, `result += stack.pop() (5) = 10`, `number = 0`, `stack = []`.
   - '**-**': `sign = -1`.
   - '**(**': push `result (10)` and `sign (-1)` to `stack`, `stack = [10, -1]`, reset `result = 0` and `sign = 1`.
   - '**6**': `number = 6`.
   - '**-**': `result = 0 + (6 * 1) = 6`, `number = 0`, `sign = -1`, .
   - '**4**': `number = 4`.
   - '**)**': `result = 6 + (4 * -1) = 2`, `result *= stack.pop() (-1) = -2`, `result += stack.pop() (10) = 8`, `number = 0`, `stack = []`.
   - **Final Calculation:** The final `result` after the loop is `8`.
3. **Return Result**:
   - The final `result` is `8`.

Code



```java
import java.util.Stack;

public class Solution {
    public int calculate(String s) {
        Stack<Integer> stack = new Stack<>();
        int result = 0; // Current result
        int number = 0; // Current number being processed
        int sign = 1;   // Current sign (1 for positive, -1 for negative)

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);

            if (Character.isDigit(c)) {
                // Building the number by multiplying the previous number by 10 and addi
                number = 10 * number + (c - '0');
            } else if (c == '+') {
                // Apply the previous sign to the number, add to the result, and reset t
                result += sign * number;
                number = 0;
                sign = 1; // Set sign as positive
            } else if (c == '-') {
```

Complexity Analysis

- **Time Complexity**: **O(n)**, where `n` is the length of the string. This is because each character in the string is processed exactly once.
- **Space Complexity**: **O(n)**, in the worst case, due to the stack used to handle parentheses. In the worst case, the stack's size is proportional to the number of parentheses, which can be as large as the length of the string.

← Previous
Basic Calculator (Hard)

Next →
Serialize And Deserialize Binary Tree (Hard)

☐ Mark as Completed